



Improving Certificate Revocation on Linux

Ubuntu Summit, May 2026

Who?



- Joe Birr-Pixton (github.com/@ctz), creator of rustls
- Dirkjan Ochtman (github.com/@djic), co-maintainer of rustls

rustls is a pure-Rust TLS protocol implementation, ten years old this year!

Problem statement

Windows:

```
C:\Users\jbp> curl https://revoked-rsa-dv.ssl.com/  
curl: (35) schannel: next InitializeSecurityContext failed:  
CRYPT_E_REVOKED (0x80092010) - The certificate is revoked.
```

Uses OS-provided certificate revocation infrastructure

Problem statement

Linux:

```
$ curl https://revoked-rsa-dv.ssl.com/  
<!DOCTYPE html>  
(...)
```

Oops! *There is no* OS-provided certificate revocation infrastructure

See also: "Linux's missing CRL infrastructure" - <https://lwn.net/Articles/1033809/>

Transport security



Transport-Level Security (TLS), formerly known as SSL:

- Confidentiality: other network participants can't understand messages
- Authentication: a way to verify who's on the other side of the line
- Integrity: messages arrive on the other side exactly as sent

Authentication in TLS

Certificate

- Binding between a public key and some names
- Signed by an issuer and time-limited

Revocation

- Certificate's issuer deciding the certificate is no longer valid

Certificate revocation list (CRL)

- Signed list of revoked certificate serial numbers

Certificate Transparency

- Browser standards require CAs submit certificates to Certificate Transparency logs

Introducing CRLite



Mozilla-built privacy-friendly synchronization protocol (started in 2020)

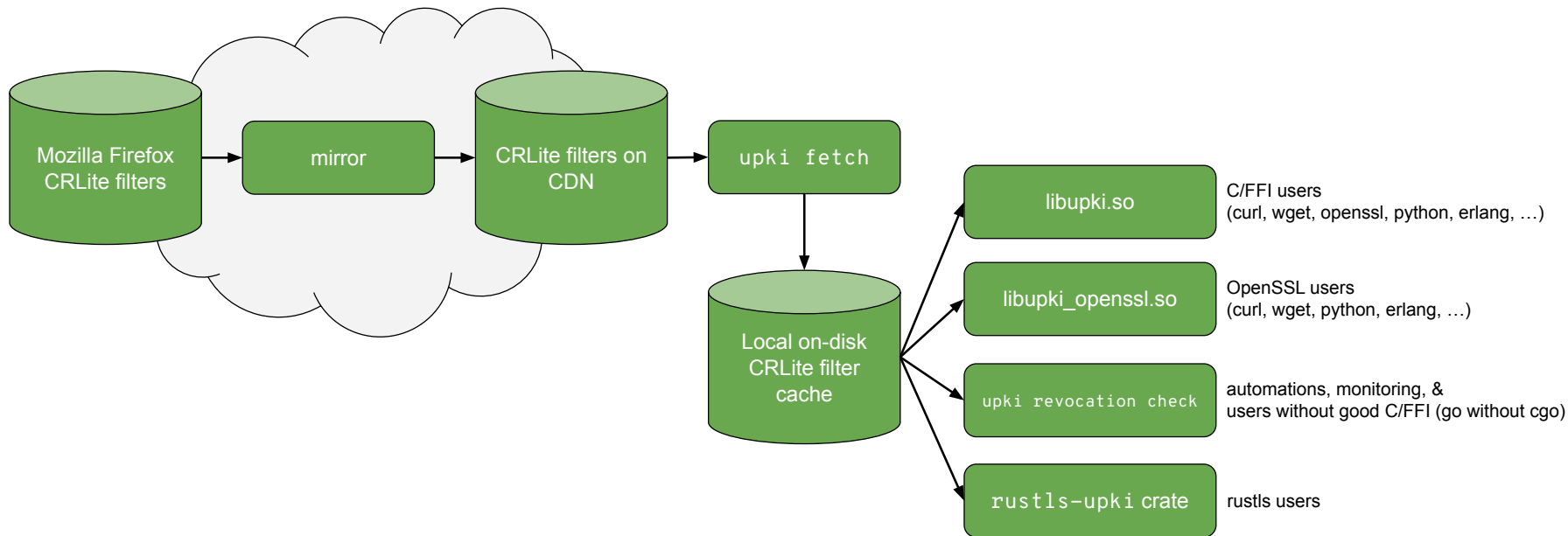
Encodes revocation status of all certificates known to Certificate Transparency logs

- Very compact (low bandwidth and storage requirements compared to a CRL)
- A bit like a bloom filter (but no false positives)
- Full CRLs are 300 MB in size; CRLite downloads 300 kB per day on average
- Checks revocation status without phoning home to the CA

Used as the main revocation tracking system in Firefox since 2025

upki packages CRLite as a library that applications on any platform can use

Packaging CRLite for Linux



Integrations



- `upki` rust crate
- `upki-rustls` rustls integration
- CLI tool (ideal for automation)
- C API (for C, C++ and FFI callers)
- OpenSSL integration
- curl patches

Demos



- Adding upki support to curl

curl patch



```
--- a/lib/vtls/openssl.c
+++ b/lib/vtls/openssl.c
@@ -56,6 +56,8 @@
     #include "curlx/base64.h"
     #endif

+#include "upki-openssl/upki-openssl.h"
+
     #include <openssl/rand.h>
     #include <openssl/x509v3.h>
     #ifndef OPENSSL_NO_DSA
@@ -3912,7 +3914,7 @@ CURLcode Curl_ossl_ctx_init(struct ossl_ctx *octx,
     * to NONE, we allow the connect to complete, regardless of the outcome.
     * We then explicitly check the result and may try alternatives like
     * Apple's SecTrust for verification. */
-    SSL_CTX_set_verify(octx->ssl_ctx, SSL_VERIFY_NONE, NULL);
+    SSL_CTX_set_verify(octx->ssl_ctx, SSL_VERIFY_NONE, upki_openssl_verify_callback);

     /* Enable logging of secrets to the file specified in env SSLKEYLOGFILE. */
     #if !defined(HAVE_KEYLOG_UPSTREAM) && defined(HAVE_KEYLOG_CALLBACK)
```

curl result



```
$ curl https://revoked-rsa-dv.ssl.com/
```

```
<!DOCTYPE html>
```

```
(...)
```

```
$ ./src/curl https://revoked-rsa-dv.ssl.com/
```

```
curl: (60) SSL certificate OpenSSL verify result: certificate revoked (23)
```

```
More details here: https://curl.se/docs/sslcerts.html
```

curl failed to verify the legitimacy of the server and therefore could not establish a secure connection to it. To learn more about this situation and how to fix it, please visit the webpage mentioned above.

upki: future goals



1. Intermediate preloading
2. Certificate transparency enforcement
(and common root distrust processes)
3. Merkle Tree Certificates

1) Intermediate preloading

Many TLS servers are misconfigured

- They don't include intermediate certificates in the TLS handshake
- Fun fact: at time of writing, example.com is misconfigured like this!
- Non-browser clients usually can't interoperate with them at all

Browsers work around this with "AIA Fetching"

- Not great for privacy, performance & architectural layering

Intermediates usable for publicly-trusted certificates are known

- upki can download them and keep them in a well-known location
- They do need to be kept up-to-date, but don't change quickly

2) Certificate Transparency support



Browsers check certificates for inclusion in CT logs

- *This aims to expose CA mis-issuance incidents*

Non-browsers don't

- Lack prior knowledge of current CT logs
 - (CT logs are introduced, retired, and fail over time)

upki will synchronise a list of current CT logs

3) Merkle Tree Certificates

Proposed replacement for certificate issuance

Which can "compress" certificates on the wire, given prior client knowledge

Especially relevant with post-quantum cryptography

- Current PQ signature algorithms use much larger signatures

Prior client knowledge - aka "Landmarks" - is another item that upki is well-placed to synchronise

Where we are today

- Canonical-maintained mirror of upki data
- Synchronization tool
 - Very easy to run from a systemd timer or cron
 - Can run in a system-wide or user-local context
- Some packaging work to do
- All described integrations
 - Please try one and give us feedback!
- Coming to Ubuntu soon

Thanks!



Repo: <https://github.com/rustls/upki>

Thanks to Canonical for supporting this work

Joe: [github.com/@ctz](https://github.com/ctz)

Dirkjan: [github.com/@djc](https://github.com/djc)
